

ПРОГРАММА ПОДГОТОВКИ СТАЖЁРОВ

Прогнозирование оттока B2B-клиентов

Рабочая тетрадь стажёра

Длительность: 4 недели

Формат: лабораторные работы + еженедельные семинары + ревью в Pull Request

Итог: мини-защита проекта

Период: август 2026

0. Как пользоваться этой тетрадью

0.1. Это главный документ курса

Всё, что нужно знать про курс, есть здесь. Остальные файлы — короткие спутники:

Файл	Что это	Когда открывать
Эта тетрадь	Полное описание курса	Постоянно. В начале недели читаете свой раздел целиком
docs/schedule.md	Только даты и время	Когда нужен дедлайн
docs/weekN_*.md	Карточка недели: те же шаги в одну страницу	Когда нужно быстро свериться в процессе работы
docs/data_dictionary.md	Только описание столбцов	Когда забыли, что значит поле
docs/review_checklist.md	Чек-листы перед PR, отдельно по неделям	Перед тем как звать ревьюера
notebooks/*.ipynb	Ваш рабочий черновик	Когда пишете код
docs/journal_<имя>.md	Ваши выводы словами	Каждый рабочий день

Если карточка недели и тетрадь расходятся — верьте тетради и напишите руководителю.

0.2. Условные обозначения

Значок	Значение
ГОТОВО ·	Уже написано в репозитории. Не переписывайте , просто используйте
ПИШЕТЕ ВЫ ·	Ваша часть работы
ВНИМАНИЕ ·	Ловушка. Здесь чаще всего ошибаются
СОВЕТ ·	Как сделать быстрее или проще
МИНИМУМ ·	Что сдать, если совсем не успеваете

0.3. Что где живёт: маршрут любого артефакта

Самый частый вопрос первой недели: «что писать в ноутбук, что в `src/`, что в журнал?» Одно правило:

Код, который вы вызываете больше одного раза, живёт в `src/`. Всё, что вы вызываете один раз, живёт в ноутбуке. Всё, что не является кодом, живёт в журнале.

Разворачиваем:

Куда	Что туда идёт	Пример
<code>src/*.py</code>	Функции: расчёт признаков, метрики, сборка выборки	<code>build_snapshot()</code> , <code>make_features()</code>
<code>notebooks/*.ipynb</code>	Вызовы этих функций, графики, таблицы метрик, <code>assert</code> -проверки	<code>dataset = build_snapshot(...)</code> , <code>plt.plot(...)</code>
<code>docs/journal_<имя>.md</code>	Решения, определения, наблюдения, числа с выводами	«Взял окно прогноза 3 месяца, потому что ...»
PR	Вся ветка целиком — и <code>src/</code> , и ноутбуки, и журнал. В свой форк, не в учебный репозиторий (см. 4.3)	—

ВНИМАНИЕ · В PR не «отправляют файл». В PR попадает всё, что вы закоммитили в ветку. Отдельно ничего прикреплять не нужно.

0.4. Что уже написано за вас

Чтобы вы не тратили время на переписывание готового:

Файл	Функция	Статус
src/eda.py	explore(df, name)	ГОТОВО ·
src/eda.py	churn_rate_by(df, target_col, group_col)	ГОТОВО ·
src/eda.py	missing_months(usage)	ПИШЕТЕ ВЫ · Неделя 1
src/features.py	trend_slope(s), months_declining(s)	ГОТОВО ·
src/features.py	window_bounds(snapshot)	ПИШЕТЕ ВЫ · Неделя 2
src/features.py	make_features(obs, obs_end)	ПИШЕТЕ ВЫ · Неделя 2 (минимум) → Неделя 3 (полная)
src/features.py	make_target(fut, alive, rev_mean)	ПИШЕТЕ ВЫ · Неделя 2
src/features.py	build_snapshot(usage, clients, snapshot)	ПИШЕТЕ ВЫ · Неделя 2
src/validation.py	precision_at_k, lift_at_k, evaluate	ГОТОВО ·
src/validation.py	bootstrap_auc_diff	ГОТОВО ·
src/validation.py	split_by_time(dataset)	ПИШЕТЕ ВЫ · Неделя 3

Если видите готовую функцию — задача не «написать такую же», а «применить и понять вывод».

0.5. Конвенция в ноутбуках

В заготовках встречаются два типа ячеек:

- **Закомментированный рабочий код** (`# explore(clients, 'Клиенты')`) — раскомментируйте и запустите. Это подсказка, а не задание.
- **# TODO: ...** — здесь пишете вы. Комментарий можно удалить.

Заготовки — черновики. Можно добавлять ячейки, менять порядок, переименовывать переменные. Нельзя одно: менять **сигнатуры функций** в `src/` (имена функций и их аргументы) — по ним ревьюер сравнивает решения между собой.

1. Что мы вообще делаем

1.1. Задача бизнеса

У телеком-оператора есть 50 000 корпоративных клиентов. Часть из них каждый месяц уходит: расторгает договор или тихо перестаёт пользоваться услугами. Удержать клиента дешевле, чем привлечь нового, но отдел удержания может обзвонить не всех, а, скажем, 5 000 компаний в квартал.

Вопрос бизнеса: каких 5 000 обзванивать?

Наш ответ: список клиентов, отсортированный по вероятности ухода.

1.2. Почему это задача ранжирования, а не классификации

Бизнесу не нужен ответ «уйдёт / не уйдёт». Ему нужен **порядок**: кого звать первым. Поэтому:

- **Не подходит:** ассигасу. Отток — редкое событие; модель «никто не уйдёт» даст точность около 90% и нулевую пользу.
- **ROC-AUC** — насколько хорошо модель упорядочивает клиентов по риску.
- **Lift@10%** — во сколько раз в топ-10% списка больше уходов, чем в случайной выборке. Это язык, который понимает бизнес.
- **Precision@10%** — какая доля из обзвоненного топ-10% реально уйдёт.

Финальный вывод проекта должен звучать так:

«В топ-10% клиентов по риску попадает X% всех будущих уходов, lift = Y.Yx. Обзвонив 10% базы, отдел удержания заставит X% уходов вместо 10% при случайном выборе.»

1.3. Главный исследовательский вопрос курса

Клиенты делятся на сегменты: малый, средний, крупный бизнес. Они ведут себя по-разному. Отсюда вопрос, на который вы отвечаете экспериментом на неделе 4:

Строить одну модель на всех или отдельную модель на каждый сегмент?

Три подхода:

- **A** — одна модель, сегмент и продукт в признаки **не** входят
- **B** — одна модель, сегмент и продукт **входят** как признаки
- **C** — три отдельные модели, по одной на сегмент

Заранее правильного ответа нет, и руководитель его вам не скажет. Ваш результат — то, что покажет эксперимент. Вывод «различия незначимы» — такой же полноценный ответ, как «подход C лучше».

2. Шесть понятий, без которых дальше нельзя

Это ядро курса. Если что-то из этого раздела непонятно — спрашивайте сразу, не откладывая: всё остальное на этом стоит.

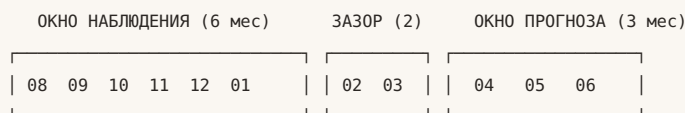
2.1. Snapshot — точка отсчёта

Мы не предсказываем «уйдёт ли клиент когда-нибудь». Мы встаём в конкретную дату прошлого, смотрим только назад и предсказываем, что случится вперёд.

Эта дата и называется **snapshot** (точка отсчёта). Например, `2025-01`.

Одна строка обучающей выборки = **один клиент в один snapshot**. Один и тот же клиент в семи снапшотах даст семь строк.

2.2. Окно наблюдения, зазор и окно прогноза



▲
snapshot = 2025-01

Отсюда берём ПРИЗНАКИ
(что клиент делал)

не используем
вообще

Отсюда берём ТАРГЕТ
(ушёл или нет)

Константы уже заданы в `src/features.py`:

```
OBSERVATION_MONTHS = 6 # окно наблюдения
GAP_MONTHS = 2 # слепой зазор
FORECAST_MONTHS = 3 # окно прогноза
```

Зачем нужен зазор. Прогноз «клиент уйдёт в следующем месяце» бизнесу бесполезен: отдел удержания не успеет ни позвонить, ни согласовать скидку, ни довести предложение до подписания. Нужен запас времени на реакцию. Зазор — это и есть формализованный запас: мы обещаем предупредить не позже чем за два месяца до события.

Есть и вторая причина, техническая: биллинг закрывает месяц не мгновенно, и данные последнего месяца в реальной системе на момент прогноза могут быть ещё неполными.

ВНИМАНИЕ · Snapshot входит в окно наблюдения, а не в зазор. Для `snapshot = '2025-01'`:

- окно наблюдения: `2024-08 ... 2025-01` (6 месяцев, включая сам snapshot)
- слепой зазор: `2025-02 ... 2025-03` (2 месяца, не используются нигде)
- окно прогноза: `2025-04 ... 2025-06` (3 месяца)

Это соглашение курса. Можно было договориться иначе, но тогда все результаты стали бы несравнимыми между студентами — поэтому делаем так.

2.3. Таргет: что считаем оттоком

В этих данных отток бывает двух видов, и это самая содержательная сложность задачи.

Явный отток. Клиент расторг договор: у него заполнено `termination_date` в `clients.xlsx`.

Тихий («спящий») отток. Договор формально не расторгнут, `termination_date` пустой, но клиент фактически перестал пользоваться услугами. Юридически он есть, практически его нет. Такие уходы справочник не видит — их придётся находить по динамике в `usage`.

Сколько таких клиентов и как именно это выглядит в данных — вы выясняете сами на неделе 1, это часть задания. Не берите на веру, посмотрите.

Рекомендованное определение таргета (можете уточнить, но обязаны зафиксировать в журнале):

Клиент считается ушедшим (`target = 1`) в данном snapshot, если выполняется **хотя бы одно** условие:

1. в окне прогноза у него **нет ни одной строки** в `usage`, **или**
2. средний `revenue` за окно прогноза составляет **менее 20%** от среднего `revenue` за окно наблюдения.

Порог 20% — отправная точка. Возьмёте другой — нормально: обоснуйте в журнале и покажите, как от него меняется доля оттока.

минимум · На **неделе 1** можно пользоваться грубым определением «ушёл = заполнен `termination_date`» — просто чтобы посмотреть на разрезы. Но запишите в журнал, что эта оценка неполная, и объясните почему.

2.4. Утечка данных (data leakage)

Утечка — это когда в признаки попадает информация, которой у вас в реальности в момент snapshot не было бы.

Проверочный вопрос для любого признака:

Мог ли я узнать это значение в день snapshot, имея на руках только базу той даты?

Если ответ «нет» — это утечка.

Четыре места, где утечка возникает чаще всего:

1. `current_status`. Это статус на момент выгрузки — то есть на конец всего периода. Если клиент числится «Расторгнут», поле будет говорить «Расторгнут» даже для снапшота двухлетней давности, когда клиент был жив и здоров. Прямое подглядывание в будущее. Использовать как признак нельзя никогда.
2. `termination_date`. То же самое. Для расчёта таргета — можно (таргет и должен быть из будущего). Как признак — нельзя.
3. **Агрегаты по всему периоду.** `usage.groupby('client_id')['revenue'].mean()` по всей таблице включает месяцы после snapshot. Признаки считаются **только по срезу окна наблюдения**.
4. **Пересечение окон между train и test.** Самая незаметная из четырёх, о ней — следующий пункт.

ВНИМАНИЕ · Главный симптом утечки: **подозрительно высокий ROC-AUC**. В реальных задачах оттока значения выше 0.95 почти всегда означают ошибку, а не гениальную модель. Увидели такое — идите искать.

Отдельно: на неделе 2 руководитель может дать вам датасет, в который утечки подсажены намеренно, и попросить их найти. Готовьтесь отвечать не «вот эта колонка выглядит странно», а «вот эта колонка, потому что её значение в день snapshot узнать было нельзя».

2.5. Валидация во времени и зазор между снапшотами

Разбивать выборку случайным перемешиванием **нельзя**: модель будет учиться на будущем и предсказывать прошлое. Разбиваем строго по времени — это называется **out-of-time**.

Но одного разбиения по снапшотам мало. Смотрите:

- train-снапшот `2025-04` берёт таргет из месяцев `2025-07 ... 2025-09`
- test-снапшот `2025-10` берёт признаки из месяцев `2025-05 ... 2025-10`

Месяцы `2025-07` ... `2025-09` попали и туда, и туда. Модель видела, чем кончились ровно те месяцы, из которых она теперь считает признаки для теста. Метрика на тесте окажется завышенной.

Лечится вторым зазором — уже не внутри снапшота, а между снапшотами: несколько снапшотов между обучающими и тестовым выбрасываем целиком. Готовая схема — в разделе 7.2, придумывать её не надо.

Не путайте два зазора: первый (2 месяца) отделяет признаки от таргета внутри одного снапшота, второй отделяет обучающие снапшоты от тестового.

2.6. «Мёртвые души»

Если клиент расторг договор ещё **до** snapshot, он не может уйти ещё раз. Такие клиенты в выборку попадать не должны — иначе доля оттока и метрики поедут.

Правило скоупа:

В snapshot попадают только клиенты, у которых есть строка в `usage` за **последний месяц окна наблюдения** (то есть за месяц самого snapshot).

Это одна строчка кода, но забывают про неё регулярно.

3. Данные

3.1. Что лежит в репозитории

Данные уже в репозитории, скачивать ничего не нужно. Датасет синтетический, но сделан по мотивам реальных биллинговых выгрузок.

Источник	Что это
<code>data/raw/clients.xlsx</code>	Справочник: одна строка = один клиент
<code>data/raw/usage/*.csv</code>	30 файлов, по одному на месяц: клиент × месяц

Точные размеры и период вы увидите сами при загрузке — это первый вопрос недели 1. Полное описание столбцов — в `docs/data_dictionary.md`.

3.2. Три поля-ловушки

Поле	Можно ли брать в признаки	Почему
<code>segment</code> , <code>region</code> , <code>product</code>	Да, джойнить свободно	Статичные атрибуты, во времени не меняются
<code>connection_date</code>	Да	Известна на любой snapshot; из неё считается «срок жизни клиента»
<code>termination_date</code>	Только для расчёта таргета	Информация из будущего
<code>current_status</code>	Никогда	Срез на конец периода, одинаковый для всех снапшотов

3.3. Как проверять данные и когда останавливаться

На неделе 1 вы будете искать в данных дубликаты, пропуски, отрицательные суммы, дыры в месяцах и прочие аномалии. Полезно заранее знать одну вещь:

Отрицательный результат — это полноценный результат.

Если вы проверили дубликаты и их нет — так и напишите: «проверил дубликаты по паре `(client_id, month)`, найдено 0 из N строк». Это правильный ответ, а не пустой. Ревьюер смотрит, умеете ли вы проверять, а не сколько грязи вы нашли.

Плохо — не проверить. Плохо — написать «дубликатов нет» без числа. Плохо — потратить полдня на поиск того, чего в данных нет, вместо того чтобы записать результат и идти дальше.

Правило по времени: если проверка не даёт находки за 15–20 минут, зафиксируйте результат и двигайтесь. Вернётесь, если позже что-то не сойдётся.

3.4. Загрузка: делается один раз

```
import glob
import pandas as pd

clients = pd.read_excel('../data/raw/clients.xlsx')          # ~15 сек, потерпите
usage_files = sorted(glob.glob('../data/raw/usage/*.csv'))
usage = pd.concat([pd.read_csv(f) for f in usage_files], ignore_index=True)

clients.to_parquet('../data/processed/clients.parquet')
usage.to_parquet('../data/processed/usage.parquet')

print(clients.shape)
print(usage.shape)
```

Дальше во всех ноутбуках читайте **только parquet**:

```
clients = pd.read_parquet('../data/processed/clients.parquet')
usage = pd.read_parquet('../data/processed/usage.parquet')
```

СОВЕТ · Excel на десятки тысяч строк читается секунд пятнадцать, parquet — доли секунды. Если вы читаете `.xlsx` в каждом ноутбуке, вы теряете по несколько минут в день и начинаете бояться перезапускать ядро — а перезапускать его надо часто.

3.5. Работа с месяцами

Месяцы записаны строками `'2025-01'`. Сравнивать их строками можно (`'2025-01' < '2025-02'` работает, потому что месяц всегда двузначный), а вот **вычитать нельзя**. Для арифметики используйте `pd.Period`:

```
import pandas as pd

snap = pd.Period('2025-01', freq='M')

obs_start = str(snap - 5)  # '2024-08' — начало окна наблюдения
obs_end   = str(snap)      # '2025-01' — сам snapshot
fc_start  = str(snap + 3)  # '2025-04' — начало окна прогноза (после зазора)
fc_end    = str(snap + 5)  # '2025-06'
```

В общем виде: `obs_start = snap - (OBSERVATION_MONTHS - 1)`, `fc_start = snap + GAP_MONTHS + 1`, `fc_end = snap + GAP_MONTHS + FORECAST_MONTHS`.

ВНИМАНИЕ · Обратите внимание на `- 5`, а не `- 6`: сам snapshot входит в окно наблюдения, поэтому назад отсчитываем на один месяц меньше. Это ошибка номер один на неделе 2, и она тихая: код работает, просто окно на месяц шире.

4. Рабочий процесс

4.1. Настройка окружения (один раз)

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

Проверка, что всё встало:

```
python -c "import pandas, sklearn, catboost, shap; print('ok')"
```

Если что-то пошло не так:

Симптом	Что делать
<code>ModuleNotFoundError</code> в Jupyter, хотя <code>pip install</code> прошёл	Jupyter запущен не из <code>venv</code> . Активируйте <code>venv</code> и запустите <code>jupyter notebook</code> из него же
В ноутбуке не то ядро	Kernel → Change kernel. Если нужного нет: <code>python -m ipykernel install --user --name churn</code>
<code>ImportError</code> на <code>src.eda</code>	Проверьте, что в ноутбуке есть <code>sys.path.append('.')</code> и что вы запускаете ноутбук из папки <code>notebooks/</code>
Ошибка при <code>to_parquet</code>	Не установлен <code>pyarrow</code> . Он есть в <code>requirements.txt</code> — переустановите зависимости

4.2. Git: ветки и коммиты

1. **Форкните** репозиторий к себе на GitHub.
2. Клонировать свой форк.
3. На каждую неделю — своя ветка:

```
git checkout main
git pull
git checkout -b week1-eda      # week2-dataset, week3-baseline, week4-experiment
```

4. Коммитьте **в конце каждого рабочего дня**, а не одним коммитом в ночь дедлайна. Ревьюер смотрит на историю: по ней видно, как шла работа.

```
git add src/ notebooks/ docs/
git commit -m "week1: профилирование таблиц + доля оттока по сегментам"
git push -u origin week1-eda
```

5. По готовности откройте на GitHub **Pull Request** — но обязательно в свой репозиторий, а не в учебный. Как именно, разбираем в следующем пункте.

4.3. Pull Request: только в свой форк

ВНИМАНИЕ · PR создаётся из вашей ветки в `main` вашего собственного форка. Не в исходный учебный репозиторий.

Это главная техническая ошибка первой недели, и она происходит по умолчанию: GitHub на странице создания Pull Request **автоматически подставляет исходный репозиторий** как базу. Если не поменять — ваш PR уедет в общий репозиторий и будет виден всем остальным студентам вместе с вашим решением.

Как должно выглядеть правильно:

```
base repository: <ваш-логин>/b2b-churn-internship   base: main
                ↑ поменять на свой!
head repository: <ваш-логин>/b2b-churn-internship   compare: week1-eda
```

Как это сделать:

1. После `git push` GitHub предложит жёлтую плашку «Compare & pull request» — нажмите её.
2. **Сразу посмотрите на левое поле `base repository`**. Если там чужой логин — нажмите на выпадающий список и выберите **свой** репозиторий.
3. Убедитесь, что `base` = `main`, а `compare` = ваша ветка недели.
4. Заголовок PR — номер и тема недели, например `Неделя 1: EDA`.
5. Создайте PR и **пришлите ссылку руководителю в чат**.

Проверка в одну секунду: в адресе открытого PR должен стоять **ваш** логин.

`https://github.com/<ваш-логин>/b2b-churn-internship/pull/1` — правильно
`https://github.com/<логин-руководителя>/b2b-churn-internship/pull/12` — неправильно

Если ошиблись и PR уже создан в чужом репозитории — просто закройте его кнопкой «Close pull request» и откройте заново с правильной базой. Ничего страшного не произошло, но лучше закрыть сразу, а не после дедлайна.

СОВЕТ · Один раз настройте это правильно — и дальше GitHub будет запоминать выбор для следующих PR в том же репозитории. Но на первом PR проверьте вручную.

ВНИМАНИЕ · Кнопка «Reviewers» на личном форке работает только для тех, у кого есть доступ к репозиторию, поэтому руководителя туда добавить не получится. Не пытайтесь — просто пришлите ссылку.

4.4. Что коммитить, а что нет

Коммитим	Не коммитим
<code>src/*.py</code>	<code>data/processed/*</code> — файлы в <code>.gitignore</code>
<code>notebooks/*.ipynb</code> с сохранённым выводом	<code>.venv/</code> , <code>__pycache__/</code>
<code>docs/journal_<имя>.md</code>	картинки, сохранённые вручную

ВНИМАНИЕ · `data/processed/dataset.parquet` в PR не попадёт — и это правильно. Данные в git не хранят. В PR идёт код, который этот файл собирает.

Проверка на воспроизводимость: удалите содержимое `data/processed/`, нажмите **Restart & Run All** — всё должно собраться заново без ошибок. Если не собирается у вас, не соберётся и у ревьюера.

4.5. Журнал

Заведите `docs/journal_<ваше_имя>.md` на первой же неделе, скопировав `docs/journal_TEMPLATE.md`. Это не формальность: половина вопросов на защите будет по журналу.

Шаблон записи:

```
## 2026-08-12

**Что делал:** посчитал долю оттока по сегментам.

**Числа:** малый X%, средний Y%, крупный Z%.

**Что решил:** разница между сегментами выглядит заметной,
проверю на неделе 4 экспериментом.

**Что непонятно / странно:** у части клиентов кривая ведёт себя
не так, как у остальных. Пока не понял, что это значит,
но записал, чтобы вернуться.
```

Последний пункт — самый ценный. Записывайте всё подозрительное, даже если не понимаете, что это. На семинаре недели 4 разбираем находки всей группы, и там это будет отдельно оцениваться.

5. Неделя 1 — Окружение и EDA

Даты: 03.08 – 09.08 · Ноутбук: `01_eda.ipynb` · Ветка: `week1-eda`

5.1. Цель одной фразой

Понять, что лежит в данных, и научиться видеть отток глазами — до того, как что-то моделировать.

5.2. Что уже готово

- **Готово** · `explore(df, name)` в `src/eda.py` — профилирует таблицу: типы, пропуски, уникальные, `describe` по числовым.
- **Готово** · `churn_rate_by(df, target_col, group_col)` в `src/eda.py` — доля оттока в разрезе группы.
- **Готово** · Ячейка загрузки данных в `01_eda.ipynb`.

Переписывать их не надо. Ваша задача — применить их и добавить одну свою функцию (шаг 3).

5.3. Пошаговый план

Шаг 1. Окружение

Поднимите `venv`, поставьте зависимости, запустите Jupyter, убедитесь, что импорты проходят (см. 4.1).

Как проверить: `python -c "import pandas, sklearn, catboost, shap; print('ok')"` печатает `ok`, и то же самое работает из ячейки ноутбука.

Шаг 2. Загрузка и `parquet`

Запустите готовую ячейку загрузки в `01_eda.ipynb`.

Как проверить: в `data/processed/` появились `clients.parquet` и `usage.parquet`, а `print(clients.shape)` и `print(usage.shape)` напечатали размеры. Запишите эти числа — они пригодятся в ответе на первый вопрос.

Шаг 3. Профилирование + одна своя функция

Раскомментируйте и запустите:

```
explore(clients, 'Клиенты')
explore(usage, 'Потребление')
```

Прочитайте вывод. Не пролистывайте — по нему вы отвечаете на половину вопросов шага 4.

пишете вы · Допишите в `src/eda.py` функцию:

```
def missing_months(usage: pd.DataFrame) -> pd.DataFrame:
    """Для каждого клиента: первый месяц, последний месяц, сколько месяцев
    фактически есть и сколько должно быть между первым и последним.

    Returns
    -----
    DataFrame с индексом client_id и столбцами:
    first_month, last_month, n_months_actual, n_months_expected, has_gap
    """
```

Смысл: у клиента могут отсутствовать месяцы **в середине** его жизни — это дырка в данных, а не уход. Отличать одно от другого важно на неделе 2.

Как найти: переведите `month` в `pd.Period`, сгруппируйте по `client_id`, возьмите `min`, `max` и `nunique`. Ожидаемое число месяцев — это разность `max - min` плюс один. Если фактическое меньше ожидаемого — есть дыра.

Как проверить: функция обрабатывает на всей таблице и печатает, у скольких клиентов `has_gap == True`. Любой результат, включая ноль, — валидный (см. 3.3).

Шаг 4. Семь вопросов

Ответьте текстом в журнале. Каждый ответ — 1–3 предложения **с числом**. Не «дубликатов нет», а «дубликатов по `(client_id, month)` нет: 0 из N строк».

№	Вопрос	По какой таблице
1	Что является одной строкой?	обе, отдельно
2	Какой ключ связывает таблицы и какая доля <code>client_id</code> из <code>usage</code> находится в <code>clients</code> (и наоборот)?	обе
3	За какой период есть данные, все ли месяцы на месте, у скольких клиентов есть дыры внутри их периода?	<code>usage</code>
4	Как в данных виден отток: сколько клиентов с <code>termination_date</code> ; сколько «пропадает» из <code>usage</code> раньше конца периода; есть ли те, у кого <code>revenue</code> упал почти до нуля без расторжения?	обе
5	Сколько клиентов в каждом сегменте, продукте, регионе?	<code>clients</code>
6	Есть ли дубликаты: по <code>client_id</code> в справочнике и по паре <code>(client_id, month)</code> в потреблении?	обе
7	Есть ли аномалии в числовых полях: отрицательные, нулевые, экстремальные выбросы?	<code>usage</code>

Вопрос 4 — самый содержательный. Первые две его части считаются в одну строчку. Третья — про тихий отток — требует подумать: у клиента нет `termination_date`, но его `revenue` давно упал. Сравните последние месяцы клиента с его же более ранним уровнем. Рецепт — в разделе 10.4.

ВНИМАНИЕ · Это **не** `docs/review_checklist.md`. Тот файл — предполётная проверка перед PR. Здесь — семь содержательных вопросов недели 1. Не путайте.

Шаг 5. Доля оттока по разрезам

На этой неделе пользуемся грубым определением: ушёл = заполнен `termination_date`.

```
clients['is_churned'] = clients['termination_date'].notna().astype(int)

churn_rate_by(clients, target_col='is_churned', group_col='segment')
churn_rate_by(clients, target_col='is_churned', group_col='product')
churn_rate_by(clients, target_col='is_churned', group_col='region')
```

ВНИМАНИЕ · Колонки `is_churned` в данных **нет**, вы создаёте её сами строкой выше. И запишите в журнале, чем эта оценка отличается от настоящей доли оттока.

Как проверить: получили три таблички с долями и количеством клиентов. Есть ли разница между сегментами? Это первая проверка главной гипотезы курса — зафиксируйте её в журнале, на неделе 4 сравните с результатом эксперимента.

Шаг 6. Графики: как выглядит уход

Постройте динамику `revenue` во времени для 5–10 ушедших и 5–10 оставшихся клиентов.

СОВЕТ · Стройте **конкретных клиентов**, а не **средние по группе**: в среднем падение размазывается и его не видно.

СОВЕТ · Ушедших выравнивайте не по календарю, а по моменту ухода: по оси X отложите «месяцев до расторжения» (от –12 до 0). Тогда все кривые падения совпадут и картинка станет читаемой. Рецепт — 10.5.

Этот график — не украшение. Посмотрев на него, вы поймёте, за сколько месяцев уход становится заметен, и это прямо повлияет на то, какие признаки вы будете строить на неделе 3. Опишите увиденное в журнале словами.

Шаг 7. Журнал

В `docs/journal_<имя>.md`: ответы на семь вопросов + 3–5 наблюдений своими словами. Наблюдение — это не число, а вывод. «Крупный бизнес уходит реже малого — вероятно, из-за длинных контрактов» — наблюдение. «6.2%» — не наблюдение.

5.4. Что должно быть в PR (Definition of Done)

- ☐ `notebooks/01_edu.ipynb` проходит **Restart & Run All** без ошибок
- ☐ `src/eda.py` содержит вашу функцию `missing_months`

- Графики динамики ушедших и оставшихся сохранены в выводе ноутбука
- `docs/journal_<имя>.md`: 7 ответов с числами + 3–5 наблюдений
- Коммитов больше одного, они распределены по дням
- PR открыт в **свой** форк (в адресе PR — ваш логин), ссылка отправлена руководителю

5.5. Пять подсказок недели 1

- Читайте `.xlsx` один раз.** Дальше только `parquet`. Иначе каждый перезапуск ядра — это лишние секунды, и вы перестанете его перезапускать, а перезапускать надо.
- Отток по справочнику — не весь отток.** `termination_date` заполнен не у всех, кто фактически ушёл. Насколько велика разница — выясните на этой неделе, это часть вопроса 4.
- `current_status` можно смотреть, но нельзя использовать.** Для разведки годится, в признаки — никогда (см. 2.4). Заведите привычку сразу: посмотрели и забыли.
- Не нашли аномалий — так и напишите с числом.** Проверка без находки — тоже работа (см. 3.3).
- Клиент, отсутствующий в ранних месяцах, ещё не подключился** (`connection_date`), а не ушёл. Проверяйте оба конца истории клиента, а не только правый.

5.6. Типичные ошибки

- Переписать `explore()` заново вместо того, чтобы применить готовую.
- Ответить на семь вопросов кодом без текста. Ревьюер читает выводы, а не только ячейки.
- Взять `current_status` «ну это же просто разведка» и потом случайно унести его в признаки на неделе 2.
- Построить средний график по всем ушедшим вместо конкретных клиентов и не увидеть ничего.

5.7. минимум , если совсем не успеваете

Шаги 1, 2, 3 (без `missing_months`), 5 и семь ответов в журнале. Графики и `missing_months` дошлёт следующим коммитом.

6. Неделя 2 — Сборка обучающей выборки

Даты: 10.08 – 16.08 · Ноутбук: `02_dataset.ipynb` · Ветка: `week2-dataset`

6.1. Цель одной фразой

Собрать таблицу `client_id | snapshot_date | признаки | segment | product | target`, в которой признаки честно из прошлого, а таргет честно из будущего.

Это самая важная неделя курса. Ошибка здесь обесценивает всё, что вы сделаете дальше: модель покажет прекрасные метрики на сломанных данных.

6.2. Порядок написания функций

`build_snapshot` опирается на три другие функции, поэтому пишутся они в таком порядке:

```
window_bounds()  ->  make_features()  ->  make_target()  ->  build_snapshot()
    границы окон      признаки из obs      таргет из будущего      всё вместе
```

На этой неделе `make_features` пишется в минимальной версии: 4 признака. Этого достаточно, чтобы конвейер заработал и все проверки прошли. На неделе 3 вы её расширяете до пяти групп — писать заново не придётся, только дописать.

Так вы получаете рабочий конвейер уже сейчас, а качеством признаков занимаетесь отдельно.

6.3. Пошаговый план

Шаг 1. Зафиксировать определение таргета

До кода. Впишите в журнал и в первую markdown-ячейку `02_dataset.ipynb`:

****Объект наблюдения:**** клиент (`client_id`). Одна строка = клиент × snapshot.

****Событие оттока:**** клиент считается ушедшим в окне прогноза, если

(а) у него нет ни одной строки в `usage` за окно прогноза, ИЛИ

(б) средний `revenue` за окно прогноза < 20% от среднего за окно наблюдения.

****Окна:**** наблюдение 6 месяцев (включая snapshot), зазор 2 месяца, прогноз 3 месяца.

****Почему так:**** порог 20% отделяет реальное затухание от сезонных колебаний; зазор в 2 месяца оставляет отделу удержания время на реакцию.

Шаг 2. `window_bounds`

ПИШЕТЕ ВЫ · Возвращает четыре строки: `obs_start`, `obs_end`, `fc_start`, `fc_end`. Через `pd.Period`, см. 3.5.

Как проверить: для `'2025-01'` должно получиться `('2024-08', '2025-01', '2025-04', '2025-06')`. Проверьте это `assert`-ом прямо в ноутбуке — тогда ошибку на месяц вы поймаете сразу, а не через три дня.

Шаг 3. Минимальная `make_features`

ПИШЕТЕ ВЫ · На вход — срез `usage` только по окну наблюдения:

```
def make_features(obs: pd.DataFrame, obs_end) -> pd.DataFrame:
    """Признаки по окну наблюдения. Индекс результата — client_id.

    Минимальная версия (неделя 2), 4 признака:
        rev_mean      — средний revenue за окно
        rev_last      — revenue последнего месяца окна
        rev_last_to_mean — rev_last / rev_mean (главный сигнал падения)
        n_months      — сколько месяцев клиент присутствует в окне
    """
```

СОВЕТ · Всё делается через `obs.groupby('client_id')`. Никаких циклов по клиентам — на миллионе строк это будет работать минутами вместо секунд.

ВНИМАНИЕ · `rev_last_to_mean` — делите на `rev_mean`, защитившись от нуля (`+ 1e-9`). Именно этот признак станет вашим baseline-правилом на неделе 3.

Как получить `rev_last`: отсортируйте по `month` и возьмите `.groupby('client_id')['revenue'].last()`, либо отфильтруйте `obs[obs['month'] == obs_end]`. Второй способ надёжнее: он не зависит от порядка строк.

Шаг 4. `make_target`

ПИШЕТЕ ВЫ · На вход — срез окна прогноза, список клиентов в скоупе (`alive`) и их `rev_mean` из окна наблюдения.

ВНИМАНИЕ · Главная ловушка недели. Клиенты, которых нет в окне прогноза, — это и есть явный отток. Если вы соедините с будущим через `merge(..., how='inner')` или `groupby` без выравнивания, эти клиенты просто исчезнут, и доля оттока получится подозрительно низкой.

Правильно: посчитайте агрегат по будущему, затем `.reindex(alive)`. Пропуски после `reindex` означают «строк нет» — то есть `target = 1`.

Как проверить: доля единиц в таргете должна быть похожа на грубую оценку недели 1 (она будет отличаться, но не в разы). Если получили 0.5% или 60% — ищите ошибку здесь.

Шаг 5. `build_snapshot`

ПИШЕТЕ ВЫ · Структура тела — ровно шесть блоков:

```
def build_snapshot(usage, clients, snapshot):
    # 1. Границы окон
    obs_start, obs_end, fc_start, fc_end = window_bounds(snapshot)

    # 2. Срез окна наблюдения
    obs = usage[(usage['month'] >= obs_start) & (usage['month'] <= obs_end)]

    # 3. Скоуп: только живые на конец окна («мёртвые души», см. 2.6)
    alive = obs.loc[obs['month'] == obs_end, 'client_id'].unique()
    obs = obs[obs['client_id'].isin(alive)]

    # 4. Признаки — ТОЛЬКО из obs
    X = make_features(obs, obs_end)

    # 5. Таргет — ТОЛЬКО из окна прогноза
    fut = usage[(usage['month'] >= fc_start) & (usage['month'] <= fc_end)]
    y = make_target(fut, alive, X['rev_mean'])

    # 6. Сборка: X + target + client_id + snapshot_date + segment/product/region
    # merge с clients делайте how='left' и потом проверяйте пропуски
```

Как проверить: соберите **один** snapshot, посмотрите на `.head(20)` и `.shape` глазами. Убедитесь, что число строк примерно равно числу активных клиентов, а не числу строк `usage`. Только после этого запускайте цикл.

Шаг 6. Сборка по семи снимкам

Канонический список — в `src/features.py`, константа `SNAPSHOTS`. **Используйте именно его**, иначе ваши результаты будут не с чем сравнивать.

```
from src.features import SNAPSHOTS

dataset = pd.concat([build_snapshot(usage, clients, s) for s in SNAPSHOTS],
                    ignore_index=True)
dataset.to_parquet('../data/processed/dataset.parquet')
```

Шаг между снимками — 3 месяца, ровно как окно прогноза: так таргеты соседних снимков не пересекаются. Крайние точки подобраны так, чтобы у первого снимка хватило истории назад, а у последнего — данных вперёд.

Шаг 7. Три обязательные проверки

```
# 1. Как ведёт себя доля оттока по снимкам
print(dataset.groupby('snapshot_date')['target'].agg(['mean', 'count']))

# 2. Ни одного пропущенного сегмента после merge
assert dataset['segment'].isna().sum() == 0

# 3. Один клиент в одном снимке — ровно одна строка
assert not dataset.duplicated(subset=['client_id', 'snapshot_date']).any()
```

Как читать первую проверку. Доля оттока по снимкам должна меняться **плавнo**. Медленный дрейф в одну сторону — нормально, так бывает и в реальных данных. А вот немонотонные скачки или разброс в разы (3% против 40%) — это баг.

Порядок поиска бага, если доля скачет: границы окон (`window_bounds`) → потеря клиентов в `make_target` → фильтр `alive` → определение таргета.

Шаг 8. Самопроверка на утечку

В `02_dataset.ipynb` — таблица по **каждому** признаку:

Признак	Из каких месяцев считается	Мог ли я знать это в день snapshot?
rev_mean	2024-08 ... 2025-01	Да
rev_last_to_mean	2024-08 ... 2025-01	Да
...		

Признак, по которому вы написали «нет» или «не уверен», в модель не идёт.

6.4. Что должно быть в PR

- ☐ `src/features.py`: работают `window_bounds`, минимальная `make_features`, `make_target`, `build_snapshot`
- ☐ `notebooks/02_dataset.ipynb` с тремя проверками и таблицей доли оттока по снимкам
- ☐ Определение таргета зафиксировано в ноутбук и в журнале
- ☐ Таблица самопроверки на утечку по всем признакам
- ☐ Ноутбук проходит Restart & Run All и **пересобирает** `dataset.parquet` с нуля

ВНИМАНИЕ · Сам `dataset.parquet` в PR не попадёт — он в `.gitignore`, и так и должно быть. Ревьюер запустит ваш ноутбук и получит файл сам.

6.5. Пять подсказок недели 2

1. **Соберите сначала ОДИН snapshot и посмотрите на него глазами.** 20 строк, `.describe()`, доля таргета. Цикл по семи снимкам запускайте только после этого — иначе будете отлаживать баг, размноженный в семь раз.
2. **Клиенты, исчезнувшие из будущего, — это и есть отток.** Не теряйте их при join'ах. Держите `alive` как источник истины по составу строк и подтягивайте к нему всё остальное через `reindex`, а не через `inner join`.
3. **Месяцы вычитаются только через `pd.Period`.** И помните про `- 5`, а не `- 6`.
4. **Не отфильтровали «мёртвые души»** — доля оттока поедет и перестанет биться с оценкой недели 1.
5. **Скачок доли оттока между снимками — всегда баг, а не «данные такие».** Порядок проверки — в шаге 7.

6.6. Типичные ошибки

- Считать признаки по всей таблице `usage`, а не по срезу окна наблюдения. Это утечка, и самая частая.
- Взять `termination_date` в признаки «просто чтобы проверить». Потом забыть убрать.
- Написать цикл `for client_id in ...`. Ждать полчаса. Решить, что данные слишком большие.
- Забыть про зазор и посчитать окно прогноза сразу после snapshot.

6.7. минимум · , если совсем не успеваете

Шаги 1–6 и первые две проверки шага 7, на трёх снимках вместо семи. Остальные снимки и таблицу утечки дошлёте. Без работающего `build_snapshot` неделя 3 не начнётся — это обязательный минимум.

7. Неделя 3 — Признаки, валидация, baseline-модели

Даты: 17.08 – 23.08 · Ноутбуки: `03_baseline.ipynb`, `04_features.ipynb` · Ветка: `week3-baseline`

7.1. Цель одной фразой

Честно сравнить три модели по возрастанию сложности — правило, логрегрессия, CatBoost — на корректной схеме валидации.

7.2. Готовая схема разбиения — не придумывайте свою

Это ключевой блок недели. Схема учитывает зазор между обучающими и тестовым снимками (см. 2.5), поэтому просто возьмите её как есть.

Снапшот	Окно наблюдения	Окно прогноза	Роль
2024-07	2024-02 ... 2024-07	2024-10 ... 2024-12	train
2024-10	2024-05 ... 2024-10	2025-01 ... 2025-03	train
2025-01	2024-08 ... 2025-01	2025-04 ... 2025-06	valid (early stopping)
2025-04	—	—	зазор, не используем
2025-07	—	—	зазор, не используем
2025-10	—	—	зазор, не используем
2026-01	2025-08 ... 2026-01	2026-04 ... 2026-06	test

Почему выброшены три снапшота. Окно наблюдения теста начинается в 2025-08. Последнее окно прогноза, которое мы используем в обучении, — 2025-04 ... 2025-06 (валидационный снапшот). Пересечения нет ни одного месяца. Если бы мы взяли в обучение снапшот 2025-04, его таргет (2025-07 ... 2025-09) лежал бы внутри признаков теста — и метрика на тесте оказалась бы завышенной.

Это цена корректной валидации: три снапшота из семи уходят в никуда. В реальных проектах платят ровно так же.

пишете вы · Оформите это в `src/validation.py`:

```
TRAIN_SNAPSHOTS = ['2024-07', '2024-10']
VALID_SNAPSHOTS = ['2025-01']
TEST_SNAPSHOTS  = ['2026-01']

def split_by_time(dataset):
    """Разбиение out-of-time с зазором. Возвращает (train, valid, test)."""
```

ВНИМАНИЕ · Тестовый период используется ровно один раз — в самом конце, когда все решения уже приняты. Если вы подбираете гиперпараметры, глядя на test, вы обучаетесь на тесте, и метрика становится враньём. Всё, что требует подбора, делается на valid.

7.3. Пошаговый план

Шаг 1. Расширить `make_features` до пяти групп

пишете вы · Дописываете к минимальной версии недели 2. Ориентировочный набор — не догма, но начните с него:

Группа	Признаки
Платежи	rev_mean, rev_min, rev_max, rev_last, rev_last_to_mean
Динамика	rev_trend (готовая trend_slope), rev_std, months_declining (готовая), traffic_last_to_mean
Объём отношений	n_sim_last, n_sim_trend, tenure_months (or connection_date до snapshot)
Сигналы боли	tickets_sum, tickets_last, debt_last, debt_months (сколько месяцев с долгом)
Категориальные	segment, product, region

ГОТОВО · `trend_slope` и `months_declining` уже написаны в `src/features.py` — примените их через `groupby(...).apply(...)`.

ВНИМАНИЕ · `tenure_months` считается до snapshot, а не до конца данных.

СОВЕТ · Признак «наклон тренда» в сыром виде пропорционален размеру клиента: у крупного бизнеса наклон в тысячах, у малого в единицах, и модель выучит размер, а не динамику. Поделите наклон на средний уровень клиента — получите относительное изменение, сравнимое между сегментами.

После расширения **пересоберите датасет** ноутбуком `02_dataset.ipynb`. Признаки живут внутри `build_snapshot`, поэтому старый `dataset.parquet` про новые колонки не знает.

Шаг 2. Baseline 0 — правило

Никакого обучения. Просто ранжируем клиентов по одному признаку:


```
from src.validation import evaluate
evaluate(y_test, -X_test['rev_last_to_mean'], 'Правило: падение платежа')
```

ВНИМАНИЕ · Минус перед признаком обязателен: чем **сильнее** упал платёж, тем **выше** риск. `evaluate` ждёт score, где больше = рискованнее. Забыли минус — получите AUC заметно ниже 0.5, и это не «модель плохая», а знак перепутан.

Этот baseline — точка отсчёта на всю неделю. Если сложная модель не обыгрывает одну колонку с минусом, бизнесу она не нужна.

Шаг 3. Baseline 1 — логистическая регрессия

Обязательно через `Pipeline` + `ColumnTransformer`:

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
```

- числовые: `SimpleImputer(strategy='median')` → `StandardScaler`
- категориальные: `OneHotEncoder(handle_unknown='ignore')`
- модель: `LogisticRegression(max_iter=1000, class_weight='balanced')`

ВНИМАНИЕ · Логрессия **без масштабирования** на признаках, где `revenue` измеряется миллионами, а `n_sim` единицами, даёт мусор. Если получили AUC около 0.5 — сначала проверьте, что `StandardScaler` реально применялся, и только потом делайте выводы о данных.

Как проверить, что пайплайн собран правильно: обучите его и посмотрите на метрику на **train**. Если и там AUC около 0.5, дело не в переобучении, а в препроцессинге.

Шаг 4. CatBoost

В `04_features.ipynb`:

```
from catboost import CatBoostClassifier, Pool

cat_cols = ['segment', 'product', 'region']
train_pool = Pool(X_train, y_train, cat_features=cat_cols)
valid_pool = Pool(X_valid, y_valid, cat_features=cat_cols)

model = CatBoostClassifier(
    iterations=2000, learning_rate=0.05, depth=6,
    eval_metric='AUC', random_seed=42, verbose=200,
)
model.fit(train_pool, eval_set=valid_pool, early_stopping_rounds=100)
```

ВНИМАНИЕ · Early stopping — на **валидационном снимоте 2025-01**, а не на случайном срезе train. Случайный срез вернёт вас к утечке, от которой мы весь курс уходим.

ВНИМАНИЕ · Категориальные столбцы у CatBoost не должны содержать `NaN` — заполните строкой `'unknown'`.

Шаг 5. Итоговая таблица и фраза для бизнеса

Сведите три модели в одну таблицу: ROC-AUC, PR-AUC, Precision@10%, Lift@10% — на **тесте**.

Чего ожидать. Обычно качество растёт от правила к логрессии и дальше к бустингу: у каждой следующей модели больше свободы. Правдоподобные значения ROC-AUC в задачах оттока лежат примерно между 0.7 и 0.95 — это широкий коридор, и точное место внутри него зависит от ваших признаков и определения таргета.

Два случая, которые надо не пропустить:

- **ROC-AUC выше 0.95** — почти наверняка утечка. Порядок поиска — в 7.5, подсказка 2.
- **CatBoost не обыграл логрессию** — это валидный результат. Проверьте одно: одинаковый ли набор признаков у обеих моделей. Если да — описывайте как есть и объясните в журнале, почему так могло получиться. Подгонять результат под ожидание нельзя; курс оценивает корректность метода.

И одна фраза в конце `04_features.ipynb` и в журнале:

«В топ-10% клиентов по риску попадает X% всех будущих уходов, $\text{lift} = Y \cdot X$. Обзвонив 10% базы, отдел удержания заставит X% уходов вместо 10% при случайном выборе.»

7.4. Что должно быть в PR

- `src/features.py` — расширенная `make_features` (пять групп)
- `src/validation.py` — `split_by_time` с константами снапшотов
- `03_baseline.ipynb` — правило и логрегрессия
- `04_features.ipynb` — CatBoost и сводная таблица метрик по трём моделям
- Фраза-вывод на языке бизнеса
- Комментарий в журнале: сошёлся ли порядок качества с ожидаемым и почему

7.5. Пять подсказок недели 3

- Правило — это не модель.** Ничего не обучайте, просто отсортируйте по признаку. Займёт две минуты и даст точку отсчёта.
- Подозрительно высокий AUC — ищите утечку, а не радуйтесь.** Порядок поиска: `current_status` / `termination_date` в признаках → признаки, посчитанные по всей таблице вместо окна → пересечение окон train и test → таргет, случайно попавший в X. Рецепт быстрой проверки — 10.7.
- Смотрите на важность признаков сразу, а не в конце.** Если один признак даёт почти всю важность, это либо утечка, либо вырожденная задача. Дёшево проверить и дорого пропустить.
- `class_weight='balanced'` и `scale_pos_weight` меняют калибровку, но почти не меняют ранжирование. А нам важно ранжирование. Не тратьте день на подбор весов классов.
- Считайте метрики на тесте один раз, в самом конце.** Всё остальное время смотрите только на valid. Каждый лишний взгляд на test — подгонка, которую потом не отследить.

7.6. минимум , если совсем не успеваете

Шаг 1 (первые три группы признаков), шаги 2, 3 и `split_by_time`. CatBoost переносится на начало недели 4 — но тогда сразу предупредите руководителя, потому что эксперимент A/B/C строится на нём.

8. Неделя 4 — Эксперимент A/B/C, интерпретация, защита

Даты: 24.08 – 30.08 · Ноутбуки: `05_segments_experiment.ipynb`, `06_final_report.ipynb` · Ветка: `week4-experiment`

8.1. Цель одной фразой

Ответить на главный вопрос курса — одна модель или по одной на сегмент — и объяснить результат на языке бизнеса.

8.2. Пошаговый план

Шаг 1. Одна функция для трёх подходов

Не копипастьте ячейки. Напишите одну функцию и вызовите её трижды:

```
def run_experiment(train, test, feature_cols, cat_cols, seed=42):  
    """Обучает модель по заданной схеме и возвращает предсказания на тесте."""
```

- A:** `feature_cols` без `segment`, `product`
- B:** `feature_cols` с `segment`, `product`
- C:** цикл по сегментам — обучаем свою модель на подвыборке сегмента, предсказываем свой кусок теста, потом **склеиваем три вектора предсказаний в один** по исходному порядку строк теста

ВНИМАНИЕ · «Одинаковые условия» — это буквально: один и тот же `dataset`, один `split_by_time`, один `seed`, одни гиперпараметры, один набор признаков (кроме того, что различается по определению подхода). Иначе эксперимент не отвечает ни на какой вопрос.

Как склеить предсказания подхода C: заведите пустой массив длиной с тест, и для каждого сегмента записывайте предсказания по булевой маске этого сегмента. Тогда порядок сохранится автоматически.

Шаг 2. Сравнение — общее и по сегментам

Таблица: строки — подходы A/B/C, столбцы — ROC-AUC и Lift@10%, отдельно **в целом** и **внутри каждого сегмента**.

ВНИМАНИЕ · Сравнение C с A и B делайте на **одном и том же полном тесте** (склеенный вектор предсказаний), а не «у каждой модели свой кусочек». Метрики, посчитанные на разных подмножествах, между собой несравнимы — это самая частая ошибка недели.

СОВЕТ · Посчитайте заодно, сколько положительных примеров попало в обучение каждой сегментной модели подхода C. Это число вам понадобится, чтобы объяснить результат.

Шаг 3. Bootstrap

ГОТОВО · `bootstrap_auc_diff(y_true, s1, s2)` уже написана и возвращает доверительный интервал разницы AUC.

Правило чтения: если интервал **накрывает ноль** — различия между подходами нет, и это полноценный ответ на исследовательский вопрос. Если не накрывает — различие есть, и тогда называйте его величину.

Формулировка «В лучше C, потому что одно число больше другого» — не результат. Формулировка «В лучше C на столько-то AUC, интервал [такой-то], различие [значимо / незначимо], для практики это [существенно / пренебрежимо]» — результат.

Шаг 4. Интерпретация: SHAP

- `model.get_feature_importance()` — топ-15 признаков
- `shap.TreeExplainer(model) + summary_plot` на подвыборке
- Разбор 1–2 конкретных клиентов из топа риска: почему модель поставила им высокий балл

СОВЕТ · SHAP на десятках тысяч строк считается долго. Берите подвыборку 2 000–5 000 строк — картинка не изменится.

ВНИМАНИЕ · Если в топе важности оказался технический идентификатор или что-то без бизнес-смысла — это сигнал утечки, а не находка.

Шаг 5. Итоговый отчёт

`06_final_report.ipynb` — десять разделов-заголовков уже расставлены. Под каждым нужны 1–3 абзаца и, где уместно, таблица или график. Отчёт должен читаться человеком, который не видел ваш код.

Финальный вывод — по шаблону:

«Подход **X** показал ROC-AUC ..., lift@10% = ...; подход **Y** — Различие [значимо / незначимо] по bootstrap. Рекомендуем **X**, потому что Основное ограничение: ...»

Шаг 6. Презентация

5–7 слайдов: задача → данные → схема сборки выборки → эксперимент A/B/C → интерпретация → вывод бизнес-языком. Кладите в `docs/` или ссылкой в README.

8.3. Что должно быть в PR

- ☐ `05_segments_experiment.ipynb` — три подхода одной функцией, таблица метрик, bootstrap
- ☐ Метрики в целом и по каждому сегменту
- ☐ SHAP summary plot + топ-15 признаков с бизнес-объяснением
- ☐ `06_final_report.ipynb` — все десять разделов заполнены
- ☐ Презентация 5–7 слайдов
- ☐ Журнал за все четыре недели

8.4. Пять подсказок недели 4

1. **Сегменты неравномерны по размеру.** Посмотрите, сколько уходов попадает в обучение каждой сегментной модели подхода C. Если в каком-то сегменте их мало, модели там будет тяжело — и это не ваша ошибка, а свойство подхода, которое вы и проверяете экспериментом. Именно такие наблюдения и есть содержательный ответ.
2. **Маленькая разница — не разница.** Без доверительного интервала не говорите «лучше».
3. **Пишите отчёт по ходу недели, а не в последний вечер.** Разделы 1–5 отчёта — пересказ того, что вы уже сделали за три недели; их можно закрыть в понедельник.
4. **Слайды делайте из отчёта, а не отдельно.** Один график = один слайд. На защите у вас 10 минут, три четверти из них — про эксперимент и вывод, а не про то, как вы читали CSV.
5. **Оставьте два-три предложения на «странные паттерны» из журнала.** Их разбирают в конце защиты, и это часто самая интересная часть. Даже нерасшифрованная находка ценится.

8.5. минимум · , если совсем не успеваете

Подходы А и В (без С), сравнение с bootstrap, feature importance без SHAP, отчёт и слайды. Подход С — сильная часть работы, но защита без него возможна; без отчёта и слайдов — нет.

9. Мини-защита

30.08, 10 минут на человека + 5 минут вопросов.

Структура выступления:

1. **Задача и определение оттока** (1 мин) — что предсказываем, какой объект, какие окна
2. **Схема сборки выборки** (2 мин) — снапшоты, зазоры, почему так
3. **Модели и метрики** (2 мин) — таблица трёх моделей, фраза на языке бизнеса
4. **Эксперимент А/В/С** (3 мин) — главный результат курса
5. **Интерпретация и ограничения** (2 мин) — топ-факторы, где выводу нельзя доверять
6. **Странные паттерны в данных** (2–3 мин, поверх регламента) — находки из журнала

Вопросы, которые прозвучат почти наверняка:

- Почему окно прогноза 3 месяца, а не 1 или 6? Почему зазор именно 2?
- Где здесь могла бы быть утечка и как вы убедились, что её нет?
- Почему три снапшота выброшены из обучения?
- Что будет с моделью через полгода без переобучения?
- Сколько денег это приносит?

СОВЕТ · Ответ «не знаю, не проверял» — нормальный, если он честный и вы понимаете, как это проверить. Ответ, придуманный на ходу, слышно сразу.

10. Как искать и как находить: рецепты

Короткие приёмы, которые закрывают 90% практических вопросов курса. Все примеры — рабочие, подставьте свои имена переменных.

10.1. Посмотреть на данные быстро

```
df.shape           # размер
df.head(20)        # первые строки
df.dtypes          # типы
df['col'].value_counts() # частоты категорий
df['col'].describe() # числовое распределение
df.isna().mean().round(3) # доля пропусков по столбцам
```

10.2. Проверить дубликаты и уникальность ключа

```
clients['client_id'].duplicated().sum() # дубли ключа справочника
usage.duplicated(subset=['client_id', 'month']).sum() # дубли пары клиент-месяц
usage.duplicated().sum() # полные дубли строк
```

10.3. Проверить, как склеиваются таблицы

```
usage['client_id'].isin(clients['client_id']).mean() # доля usage, нашедшая клиента
clients['client_id'].isin(usage['client_id']).mean() # доля клиентов, имеющих потребление
```

Оба числа близки к 1 — таблицы согласованы. Сильно меньше — разбирайтесь, кто лишний.

10.4. Найти клиентов с падением без расторжения

Идея: сравнить недавний уровень клиента с его же более ранним уровнем.

```
recent = usage[usage['month'] >= 'ГГГГ-MM'].groupby('client_id')['revenue'].mean()
earlier = usage[(usage['month'] >= 'ГГГГ-MM') &
                 (usage['month'] <= 'ГГГГ-MM')].groupby('client_id')['revenue'].mean()

ratio = (recent / earlier).dropna()
suspects = ratio[ratio < 0.2].index      # порог подберите сами
```

Дальше пересеките `suspects` с теми, у кого `termination_date` пустой. Посмотрите на несколько таких клиентов графиком — убедитесь, что это действительно затухание, а не разовый провал.

10.5. Выворачивать графики по событию, а не по календарю

```
term = clients.dropna(subset=['termination_date'])[['client_id', 'termination_date']]
m = usage.merge(term, on='client_id')
m['k'] = (m['month'].map(lambda x: pd.Period(x, 'M')) -
         m['termination_date'].map(lambda x: pd.Period(x, 'M'))).map(lambda x: x.n)
# теперь k = 0 в месяц расторжения, -1 месяцем раньше и т.д.
```

Постройте `revenue` против `k` для нескольких клиентов — кривые падения совпадут по фазе, и станет видно, за сколько месяцев уход становится заметен.

СОВЕТ · Чтобы сравнивать клиентов разного размера на одном графике, поделите `revenue` каждого клиента на его собственный уровень за ранние месяцы. Тогда все кривые начнутся примерно с единицы.

10.6. Отладить границы окон

Первым делом — `assert` на известный пример:

```
assert window_bounds('2025-01') == ('2024-08', '2025-01', '2025-04', '2025-06')
```

Если не сходится, печатайте промежуточные значения `pd.Period` по одному. Ошибка почти всегда в одном из двух мест: смещение на единицу (`- 5` против `- 6`) или забытый `GAP_MONTHS`.

Вторая проверка — на составе месяцев в срезе:

```
sorted(obs['month'].unique()) # должно быть ровно 6 месяцев
sorted(fut['month'].unique()) # должно быть ровно 3 месяца
```

10.7. Быстро проверить, нет ли утечки

Три приёма, по возрастанию затрат:

1. **Глазами по списку признаков.** Для каждого — вопрос из 2.4. Занимает пять минут.
2. **По важности признаков.** Обучите модель, посмотрите топ-5. Если один признак забирает почти всю важность — открывайте его и проверяйте, из каких месяцев он считается.
3. **По одиночному AUC.** Посчитайте `roc_auc_score(y, X[col])` для каждого признака отдельно. Значение выше 0.9 или ниже 0.1 у одной колонки — сильный повод присмотреться. Ниже 0.1 значит «идеальный предиктор с обратным знаком», это тоже находка.

10.8. Не потерять клиентов при соединении

```
before = len(alive)
result = features.reindex(alive)      # HE merge(how='inner')
assert len(result) == before
assert result.index.equals(pd.Index(alive))
```

Правило: заранее решите, какой список клиентов является правильным составом строк, и после каждого соединения проверяйте, что состав не изменился.

10.9. Ускорить медленный код

Медленно	Быстро
<code>for client_id in df['client_id'].unique():</code>	<code>df.groupby('client_id').agg(...)</code>
<code>df.apply(func, axis=1)</code>	векторные операции над столбцами
<code>pd.read_excel</code> / <code>read_csv</code> в каждом ноутбуке	один раз, потом <code>read_parquet</code>
<code>df = df.append(...)</code> в цикле	собрать список и <code>pd.concat</code> один раз

Если ячейка считается дольше минуты — почти всегда дело в цикле по клиентам.

10.10. Куда смотреть, когда непонятно

Вопрос	Где ответ
Что значит поле	<code>docs/data_dictionary.md</code>
Что делать на этой неделе	эта тетрадь, разделы 5–8; кратко — <code>docs/weekN*.md</code>
Что должно быть в PR	«Что должно быть в PR» в разделе недели + <code>docs/review_checklist.md</code>
Как называется функция и что она принимает	docstring в <code>src/*.py</code>
Что вообще происходит	раздел 2 этой тетради
Почему у меня цифра не такая	раздел 11

Если ответа нет ни в одном из этих мест — пишите руководителю. Хороший вопрос: «делаю то-то, ожидал такое, получил такое, проверил вот это». Плохой вопрос: «не работает».

11. Диагностика: симптом → что проверять

Симптом	Вероятная причина	Что делать
Доля оттока скачет между снимками в разы	Границы окон или потеря клиентов	Проверить <code>window_bounds</code> ассертом (10.6), затем состав строк после соединений (10.8)
Доля оттока подозрительно низкая	Клиенты без строк в будущем потерялись	<code>reindex(alive)</code> вместо <code>inner join</code> в <code>make_target</code>
Доля оттока подозрительно высокая	Не отфильтрованы «мёртвые души»	Оставить только клиентов со строкой за месяц <code>snapshot</code> (2.6)
ROC-AUC выше 0.95	Утечка	Три приёма из 10.7
ROC-AUC около 0.5 у логрессии	Не применилось масштабирование	Проверить <code>ColumnTransformer</code> ; посмотреть метрику на <code>train</code>
ROC-AUC заметно ниже 0.5 у правила	Перепутан знак	Поставить минус перед признаком
CatBoost хуже логрессии	Разные наборы признаков или ошибка в <code>early stopping</code>	Сверить списки колонок; проверить, что <code>eval_set</code> — валидационный снимок
Метрики A/B/C несравнимы	У подхода C предсказания считались на разных подмножествах	Склеить три вектора в один по маскам сегментов (8.2, шаг 1)
<code>assert</code> на дубли клиент × снимок падает	В <code>alive</code> попал клиент дважды	Проверить <code>unique()</code> ; проверить дубли в исходном <code>usage</code>
Ячейка считается минутами	Цикл по клиентам	Переписать через <code>groupby</code> (10.9)
<code>Restart & Run All</code> падает, хотя по ячейкам всё работало	Порядок ячеек или переменная из удалённой ячейки	Перезапустить с нуля и идти сверху вниз до первой ошибки
Ревьюер не видит ваш файл	Файл в <code>.gitignore</code> или не закоммичен	<code>git status</code> ; данные коммитить не надо, код — надо

12. FAQ

«В ноутбуке написано одно, в карточке недели другое. Чему верить?» Этой тетради. Ноутбуки и карточки — короткие спутники, их текст сжат. Расхождение — сообщите, поправим.

«Функция уже написана в файле. Что мне добавлять?» Ничего. Раздел 0.4 — полная таблица того, что готово, а что пишете вы. Готовые функции нужно применить и понять их вывод.

«Что именно отправлять в PR?» Ничего отдельно отправлять не нужно. Всё, что вы закоммитили в ветку недели, попадёт в PR автоматически. Прислали ссылку — сдали.

«В какой репозиторий делать Pull Request?» В свой собственный форк, из ветки недели в `main` вашего же репозитория. GitHub по умолчанию подставляет учебный репозиторий — это надо поменять вручную на первом PR. Пошагово с картинкой полей — 4.3.

«Что писать в `src/`, а что в ноутбук?» Вызывается больше одного раза → `src/`. Один раз → ноутбук. Не код → журнал (0.3).

«Семь вопросов — это те, что в `review_checklist.md`?» Нет. Семь вопросов недели 1 — в разделе 5.3, шаг 4. `review_checklist.md` — предполётная проверка перед PR, она разбита по неделям.

«Мне нужен `make_features` для недели 2, а он выглядит как задача недели 3.» На неделе 2 пишете минимальную версию (4 признака, 6.3 шаг 3), на неделе 3 расширяете. Заново переписывать не придётся.

«Зачем зазор между окном наблюдения и окном прогноза?» Чтобы прогноз был полезен: отдел удержания должен успеть отреагировать. Подробно — 2.2.

«У меня ROC-AUC 0.98. Я молодец?» Почти наверняка нет. См. 7.5, подсказку 2, и рецепт 10.7.

«У меня CatBoost хуже логрегрессии. Я всё сломал?» Не обязательно. Проверьте одинаковость признаков — и если всё так, описывайте результат как есть. Курс оценивает корректность метода, а не совпадение с ожидаемым порядком.

«Ничего не успеваю.» У каждой недели есть блок «минимум». Сделайте его и напишите руководителю **заранее**, а не в день дедлайна.

«Где `docs/journal_<имя>.md`? Его нет в репозитории.» Скопируйте `docs/journal_TEMPLATE.md` и подставьте своё имя: `docs/journal_ivanov.md`.

«Можно менять окна, снапшоты или порог тихого оттока?» Окна и список снапшотов — нет, иначе решения несравнимы. Порог тихого оттока — можно, если обоснуете в журнале и покажете, как от него меняется доля оттока.

13. Глоссарий

Термин	Что означает
Snapshot	Точка отсчёта. Дата в прошлом, из которой мы смотрим назад (признаки) и вперёд (таргет)
Окно наблюдения	6 месяцев до snapshot включительно. Единственный источник признаков
Зазор (gap)	2 месяца сразу после snapshot. Не используются нигде
Окно прогноза	3 месяца после зазора. Единственный источник таргета
Таргет	1, если клиент ушёл в окне прогноза (явно или тихо), иначе 0
Явный отток	Расторжение договора: заполнен <code>termination_date</code>
Тихий отток	Договор не расторгнут, но клиент фактически перестал пользоваться услугами
Мёртвые души	Клиенты, ушедшие до snapshot. В выборку не попадают
Data leakage	Признак содержит информацию, недоступную в момент snapshot. Завышает метрики, ломает модель в проде
Out-of-time	Разбиение train/test по времени, а не случайно
Embargo	Выброшенные снапшоты между train и test, чтобы окна не пересекались
ROC-AUC	Насколько хорошо модель упорядочивает клиентов по риску. 0.5 — случайно, 1.0 — идеально
PR-AUC	То же для редкого класса; чувствительнее к дисбалансу
Precision@10%	Доля реально ушедших среди топ-10% списка
Lift@10%	Во сколько раз топ-10% списка «богаче» уходами, чем случайные 10%
Bootstrap	Многократное пересэмплирование выборки, чтобы оценить разброс метрики

14. Чек-листы по неделям

Проходите по своему списку **до** того, как звать ревьюера. Дублируются в `docs/review_checklist.md`.

Неделя 1

- ☐ Restart & Run All проходит
- ☐ `explore()` применена к обеим таблицам, вывод прочитан
- ☐ `missing_months()` написана и работает
- ☐ Семь ответов в журнале — каждый с числом
- ☐ Доля оттока посчитана по сегменту, продукту, региону
- ☐ Графики revenue для ушедших и оставшихся построены
- ☐ 3–5 наблюдений словами в журнале
- ☐ Коммитов больше одного
- ☐ PR открыт **в свой форк**, а не в учебный репозиторий — в адресе PR ваш логин

Неделя 2

- ☐ Restart & Run All проходит и **пересобирает** `dataset.parquet` с нуля
- ☐ Определение таргета зафиксировано в ноутбуке и журнале
- ☐ `window_bounds`, `make_features`, `make_target`, `build_snapshot` работают
- ☐ Собраны все семь снапшотов из `SNAPSHOTS`
- ☐ Доля оттока по снапшотам меняется плавно — таблица приложена
- ☐ Оба `assert` проходят
- ☐ Таблица самопроверки на утечку заполнена по всем признакам
- ☐ Ни `current_status`, ни `termination_date` в признаках нет

Неделя 3

- ☐ Restart & Run All проходит в обоих ноутбуках
- ☐ `make_features` покрывает пять групп признаков
- ☐ Датасет пересобран после расширения признаков

- ☐ `split_by_time` реализует схему с зазором из 7.2
- ☐ Есть все три модели: правило, логрегрессия, CatBoost
- ☐ Early stopping — на валидационном снапшоте, не на случайном срезе
- ☐ Метрики ранжирующие, не ассигасу
- ☐ Проверено, нет ли утечки; если AUC высокий — объяснено, почему
- ☐ Тест использован один раз, в конце
- ☐ Фраза-вывод на языке бизнеса написана

Неделя 4

- ☐ Три подхода прогнаны одной функцией, условия идентичны
- ☐ Предсказания подхода C склеены в один вектор по полному тесту
- ☐ Метрики в целом и по каждому сегменту
- ☐ Bootstrap проведён, вывод сформулирован через интервал
- ☐ SHAP summary + топ-15 признаков с бизнес-объяснением
- ☐ Все десять разделов итогового отчёта заполнены
- ☐ Презентация 5–7 слайдов готова
- ☐ Журнал заполнен за все четыре недели
- ☐ Ограничения работы названы явно

Вопросы, расхождения и находки — руководителю в чат, не копите до дедлайна.